FUN GC ／ UNITY講座シリーズ

【Unity基礎】

オブジェクト指向をC#で学ぼう

第3回　クラスを受け継ぐ「継承」



2026年10月26日　講師：嘉屋悠月

TODAY・今日のゴール

3つできるようになろう



親にまとめる

共通の部分を、親の Enemy に1回だけ書ける



違いだけ書ける

Slime と Boss を「違う部分だけ」で作れる



見せ方を決める

override・abstract・protected で、子との関係を決められる

POINT　用語は後から。まず動かして体験してから、名前を付けよう。

TODAY・90分

今日の流れ

01	継承の基本	15分
02	上書き・base・abstract	20分
03	カプセル化の復習と protected	20分
04	演習：Slime と Boss をつくる	30分
05	まとめ・次回予告	5分

RECAP ・ 前回のふりかえり

前回作った Enemy、覚えてる？

第2回のコードから

```
public class Enemy
{
    private int hp = 100;

    public void TakeDamage(int damage)
    { hp -= damage; … }
}
```

データは private、
入口はメソッド。

第2回で、外から壊されない Enemy を作った。
では、この Enemy から
Slime や Boss も作りたくなったら？
Enemy を何回もコピーする？

SECTION

01

継承の基本

親クラスの変数やメソッドを、
子クラスがそのまま受け継ぐしくみ



hp も Move() も、毎回コピーして書く？

Slime.cs

```
public class Slime
{
    int hp = 10;
    void Move() { … }
    void Jump() { … }
}
```

Boss.cs

```
public class Boss
{
    int hp = 10;
    void Move() { … }
    void Fire() { … }
}
```

? **敵が10種類になったら、同じ hp と Move() を10回書く。直すときも10か所。……もっといい方法がありそう？**

子は親の「持ち物」と「動き」を受け継ぐ

親クラス（基底クラス）

変数やメソッドをまとめて持っておく。
共通の部分を一か所にまとめられる。

子クラス（派生クラス）

親を受け継ぎ、子だけの機能を足す。
受け継いだ分は書かなくてよい。

書き方は「:」

class 子 : 親 と書くと、
その子は親を受け継ぐ。



class 子 : 親 で受け継げる

Enemy.cs ／ Slime.cs

```
public class Enemy
{
    protected int hp = 10;
    public void Move() { Debug.Log("移動する"); }
}

public class Slime : Enemy
{
    // hp と Move() は Enemy から受け継ぐ
    public void Jump() { Debug.Log("ぴょん！"); }
}
```

① 「:」で親を指定

Slime : Enemy と書くと、
Slime は Enemy の子になる。

② 親の中身はそのまま使える

hp と Move() は書かなくても、
Slime が持っている。（hp の protected は 03 で説明）

③ 子だけの機能を足す

Jump() は Slime だけがもつ。
Enemy には増えない。

SECTION

02

上書き・base・abstract



子の動きに差し替える「上書き」と、中身のない約束

上書きは override、親の処理は base

Enemy.cs ／ Boss.cs

```
public class Enemy
{
    public virtual void Attack()
    {
        Debug.Log("体当たり");
    }
}
public class Boss : Enemy
{
    public override void Attack()
    {
        base.Attack(); // 親の Attack() も実行
        Debug.Log("ファイアを放つ！");
    }
}
```

virtual（親）

「上書きしてOK」と
親が宣言する。

override（子）

同じ名前・同じ形の動きを
子で置き換える。

base.Attack()

親の Attack() を呼び出す。
親の処理 → 子の処理の順に動く。

abstract は「中身のない約束」

Enemy.cs ／ Slime.cs

```
public abstract class Enemy
{
    public abstract void Attack(); // 中身（{ }）はない
}

public class Slime : Enemy
{
    public override void Attack()
    {
        Debug.Log("ぷよっ！");
    }
}

// new Enemy(); → エラー（abstract は new できない）
```

① abstract クラス

親専用の設計図。
new で作ることはできない。

② abstract メソッド

中身（{ }）は書かず、形だけ決める。
virtual と違い、子は必ず override。

③ 書き忘れると

子で Attack() を書かないと
コンパイルエラーになる。

SECTION

03

カプセル化の復習と protected



第2回の private を思い出して、子にだけ見せる方法を学ぼう

アクセス修飾子で「どこから見えるか」

カプセル化は第2回の内容。データを隠し、入口（メソッド・プロパティ）から守るのが目的です。

修飾子	同じクラス	子クラス	外部	役割
private	○	×	×	完全に隠す（データの基本）
protected	○	○	×	子にだけ開く（今日のポイント）
public	○	○	○	誰にでも開く（入口に使う）

第2回の合言葉：データは private、さわり方は public なメソッドから。
protected は「子にだけ開く入口」。

子から hp を触ろうとすると、エラー

Enemy.cs ／ Slime.cs

```
public class Enemy
{
    private int hp = 10;
}

public class Slime : Enemy
{
    public void Hurt()
    {
        hp -= 1; // ← ここでエラー
    }
}
```

Console

```
error CS0122:
'Enemy.hp' is inaccessible
due to its protection level
```

? **Slime は Enemy の子なのに、hp が使えない。private は「そのクラスの中だけ」だから。子にだけ見せる方法は？**

つまずき：子からは private が見えない

⚠ NG　子からは使えない

```
private int hp;
hp = 5;
```

private は持ち主のクラスの中だけ。
子クラス（Slime）からは見えない。

✓ OK　子からも使える

```
protected int hp;
hp = 5;
```

protected は、子クラスからも使える。
外の無関係なクラスからは使えない。

迷ったら private。子に見せたいものだけ protected にする。

子にだけ開く：protected

Enemy.cs ／ Slime.cs ／ GameManager.cs

```
public class Enemy
{
    protected int hp = 10;
}

public class Slime : Enemy
{
    public void Hurt() { hp -= 1; } // OK
}

// 別のクラス（外）から
slime.hp = 99; // エラー
```

① protected ＝ 子にだけ開く

自分のクラスと、その子クラスの中から使える。

② 外からは今までどおり見えない

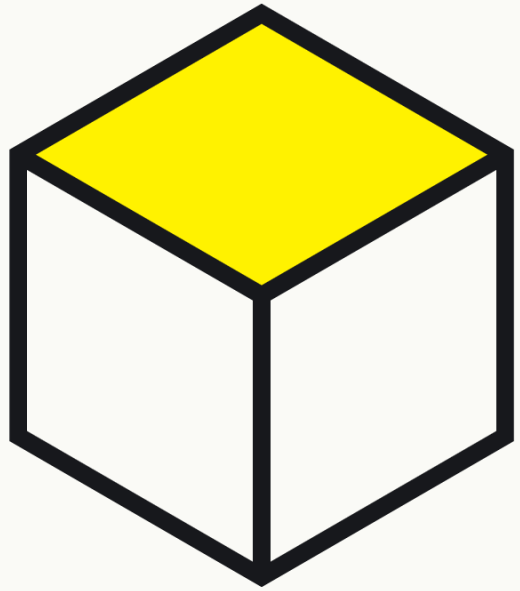
まったく関係ないクラスからは、private と同じで使えない。

③ 入口はこれまでどおり

外から変えさせたいときは、public なメソッドを通す（第2回）。

POINT ここだけは覚える

「〜の一種」なら、継承を使う。



例：Slime は Enemy の一種　→　継承

例：Enemy は Weapon を持っている　→　変数

EXERCISE

演習

目安時間

30分

Slime と Boss をつくろう

ゴール　Enemy を親にして、
Slime と Boss で Attack() の動きを変える

1. Enemy.cs：protected な hp と virtual な Attack() を書く
2. Slime.cs：Attack() を override して表示を変える
3. Boss.cs：base.Attack() を呼び、「ファイア！」を足す
4. Start() で new して Attack() を呼び、Console を確認

早く終わった人へ

Slime に Jump() を足す ／ override を外して CS0114 の警告を見る

今日できるようになったこと



継承の書き方

class 子 : 親 と書くと、
親の変数とメソッドを受け継ぐ



上書きと abstract

abstract は中身のない約束。
子が必ず実装する。
base で親の処理も呼べる



使いどころ

「〜の一種」なら継承。
子に見せるなら protected。

NEXT **第4回　ポリモーフィズムとインターフェース**

同じ呼び方なのに、動きが変わる。敵をまとめて動かすしくみを学びます。